



Εργασία στο μάθημα Big Data

Πετρούλα Τσιμπίνη ics23085

ΘΕΜΑ 1

Στο πρώτο θέμα της εργασίας χρησιμοποιήσαμε το Spark RDD API ,προκειμένου να αναλύσουμε τα δεδομένα και να υπολογίσουμε την μέση καθυστέρηση ανά αεροδρόμιο αναχώρησης και να επιστρέψουμε τα top-10 αεροδρόμια με την μεγαλύτερη μέση καθυστέρηση.

Σταδιακά εκτελέσαμε τα εξής:

1)Διαχείριση Δεδομένων: Διαβάσαμε το αρχείο flights_2000.csv ως Text RDD. Η `parse_line` συνάρτηση χρησιμοποιεί τον `csv.reader` για ασφαλές parsing, εξάγοντας τα πεδία Origin Airport, Departure Delay και Cancelled.

2)Φιλτράρισμα: Εξαιρέσαμε τις ακυρωμένες πτήσεις (`CANCELLED == 1`) και όσες δεν έχουν καταγεγραμμένη καθυστέρηση.

3)Υπολογισμός Μέσου Όρου: Δημιουργήσαμε ένα Pair RDD (airport, delay). Ο υπολογισμός της μέσης καθυστέρησης ανά αεροδρόμιο έγινε με τη χρήση της μεθόδου `combineByKey`, η οποία συσσωρεύει το (άθροισμα καθυστερήσεων,πλήθος πτήσεων).

4)Top-10 Αποτελέσματα: Μετατρέψαμε το RDD σε `(average_delay,airport)`,ταξινόμησαμε σε φθίνουσα σειρά (`sortByKey(False)`) και εκτυπώσαμε τα 10 αεροδρόμια με τη μεγαλύτερη μέση καθυστέρηση (`take(10)`).

Για να χρονομετρήσουμε τρέξαμε τον κώδικα 5 φορές και καταγράψαμε:

1)Τον χρόνο του πρώτου action (για να ξεπεραστεί η Lazy Evaluation).

2)Τον συνολικό χρόνο εκτέλεσης.

Υπολογίσαμε τον τελικό μέσο χρόνο από τις τρεις ενδιάμεσες τιμές (αφαιρώντας τη μεγαλύτερη και τη μικρότερη) για πιο αξιόπιστη μέτρηση.

Πίνακας Αποτελεσμάτων (RDD API)

Αεροδρόμιο	Μέση Καθυστέρηση (λεπτά)
DFW	11.98
JFK	11.20
SEA	9.56
ORD	9.35
LAS	9.11
CLT	8.82
MCO	8.69
BOS	8.63
DEN	8.44
MIA	8.26

Πίνακας Χρόνων Εκτέλεσης

Εκτέλεση	Συνολικός Χρόνος (sec)	Χρόνος Πρώτου Action (sec)
1	1.5394	0.4083
2	1.4089	0.3306
3	1.3853	0.2727
4	1.3119	0.2909
5	1.3065	0.3128

--- Μέσοι Χρόνοι ---

Μέσος Συνολικός Χρόνος	**1.3687 sec**
Μέσος Χρόνος Πρώτου Action	**0.3114 sec**

ΘΕΜΑ 2

Στο δεύτερο θέμα της εργασίας, χρησιμοποιήθηκε το Spark DataFrame API για την ανάλυση των δεδομένων πτήσεων. Συγκεκριμένα, η ανάλυση βασίστηκε στον υπολογισμό της μέσης καθυστέρησης αναχώρησης ανά διαδρομή (συνδυασμός αεροδρομίου αναχώρησης και προορισμού) και στην επιστροφή των Top-10 διαδρομών με τη μεγαλύτερη μέση καθυστέρηση.

Σταδιακά εκτελέσαμε τα εξής:

1)Καθαρισμός Δεδομένων: Διαβάσαμε το αρχείο flights_2000.csv ως DataFrame. Κάναμε φιλτράρισμα για την εξαίρεση των **ακυρωμένων** πτήσεων (CANCELLED = 0) και των γραμμών με κενές τιμές στην καθυστέρηση αναχώρησης (DEP_DELAY).

2)Υπολογισμός Top-10 Διαδρομών: Δημιουργήσαμε την στήλη ROUTE (origin-destination). Η μέση καθυστέρηση υπολογίζεται με groupBy("ROUTE") και agg(F.avg("DEP_DELAY")). Ταξινομήσαμε κατά φθίνουσα σειρά και επιλέξαμε τις 10 κορυφαίες διαδρομές (limit(10))τις οποίες αποθηκεύσαμε στο top_routes_delays.csv.

Η χρονομέτρηση πραγματοποιήθηκε με τον ίδιο τρόπο όπως και στο θέμα 1.

Δείγμα Εξόδου (Top-10 Routes)		
Route	Μέση Καθυστέρηση (λεπτά)	
DFW-JFK	23.30	
JFK-LAS	22.80	
MIA-JFK	20.50	
MCO-ORD	19.82	
MCO-LAX	19.14	
BOS-SEA	18.86	
DFW-ATL	18.62	
BOS-PHX	18.18	
JFK-MIA	18.00	
DEN-SFO	17.86	

Πίνακας Χρόνων Εκτέλεσης (DataFrame API)		
Εκτέλεση	Συνολικός Χρόνος (sec)	Χρόνος Πρώτου Action (sec)
1	1.2212	1.0115
2	0.2209	0.2891
3	0.1213	0.2303
4	0.0955	0.2240
5	0.1454	0.1686

```

--- Μέσοι Χρόνοι (3 ενδιάμεσες εκτελέσεις) ---
| **Μέσος Συνολικός Χρόνος** | **0.1625 sec** |
| **Μέσος Χρόνος Πρώτου Action** | **0.2478 sec** |

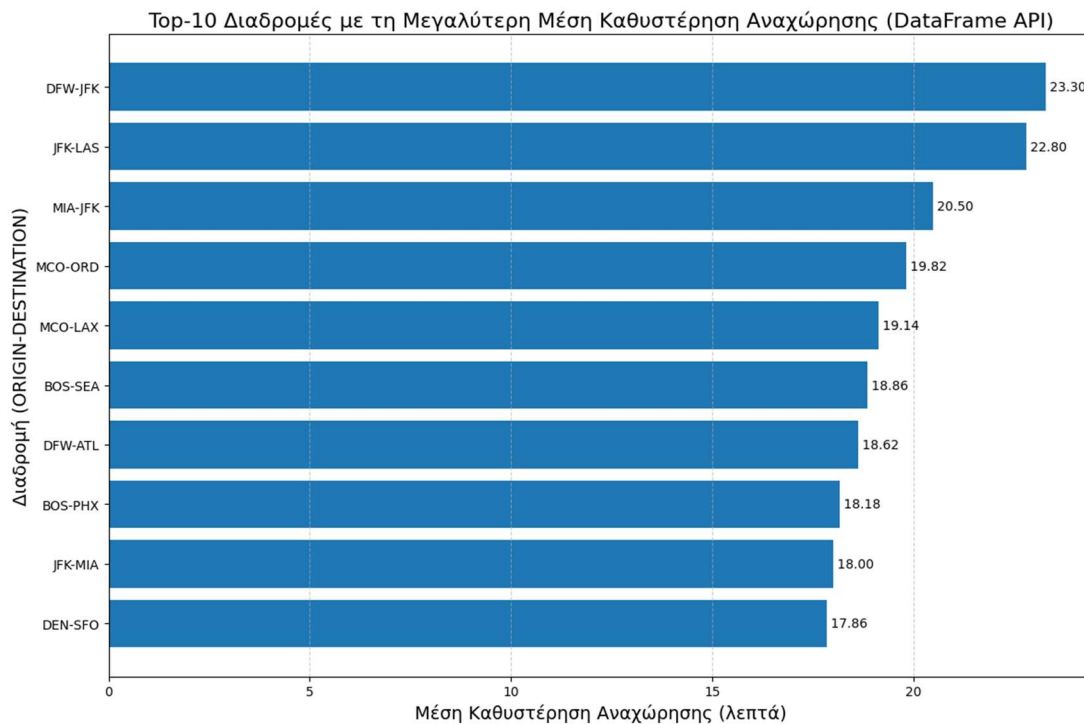
```

ΘΕΜΑ 4,5:Οπτικοποίηση

Για την οπτικοποίηση, τα αποτελέσματα του Spark DataFrame Api τα μετατρέψαμε σε Pandas DataFrame μέσω του action toPandas().

1)Top-10 Διαδρομές

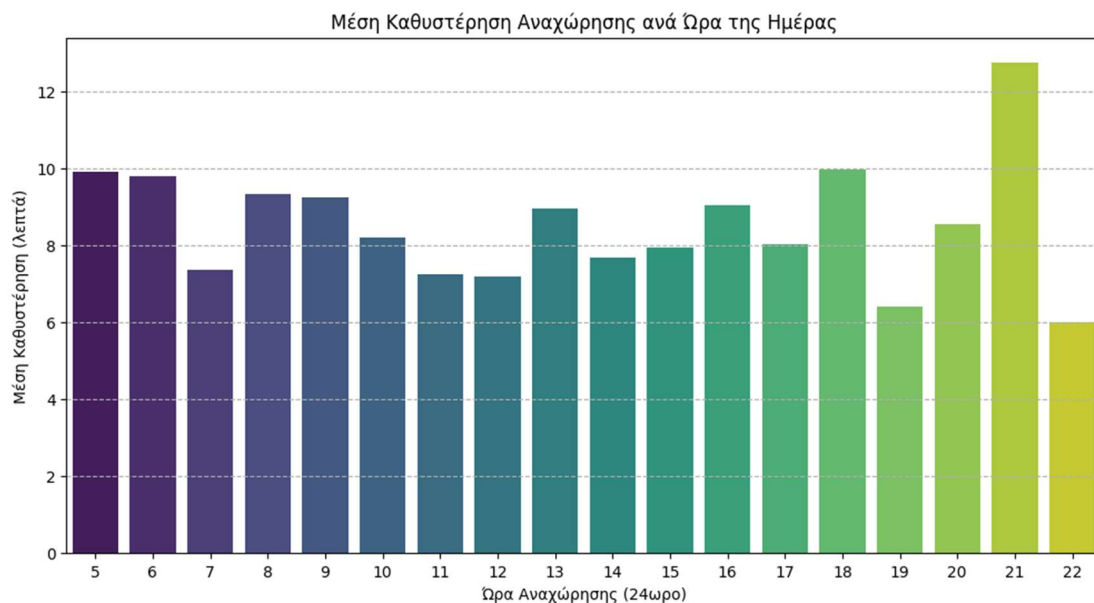
Δημιουργήσαμε ένα οριζόντιο ραβδόγραμμα (matplotlib.pyplot.barh) για την εμφάνιση των top-10 διαδρομών με τη μεγαλύτερη μέση καθυστέρηση. Η οπτικοποίηση αναδεικνύει ποιες διαδρομές αντιμετωπίζουν τα σοβαρότερα προβλήματα καθυστέρησης αναχώρησης.



2) Ανάλυση Καθυστέρησης ανά Ώρα της Ημέρας

Ο σκοπός του γραφήματος είναι να εντοπιστεί η χρονική στιγμή της ημέρας (ώρα) κατά την οποία οι καθυστερήσεις αναχώρησης είναι οι μεγαλύτερες.

Δημιουργήσαμε ένα ραβδόγραμμα (`seaborn.barplot`) για να δείξουμε τη μέση καθυστέρηση για κάθε ώρα της ημέρας. Δημιουργούμε τη στήλη `HOUR` (24ωρο) από το πεδίο `SCHED_DEP` (`F.substring(..., 12, 2)`) και υπολογίζουμε τη μέση καθυστέρηση (`AvgDelay`) ανά ώρα.



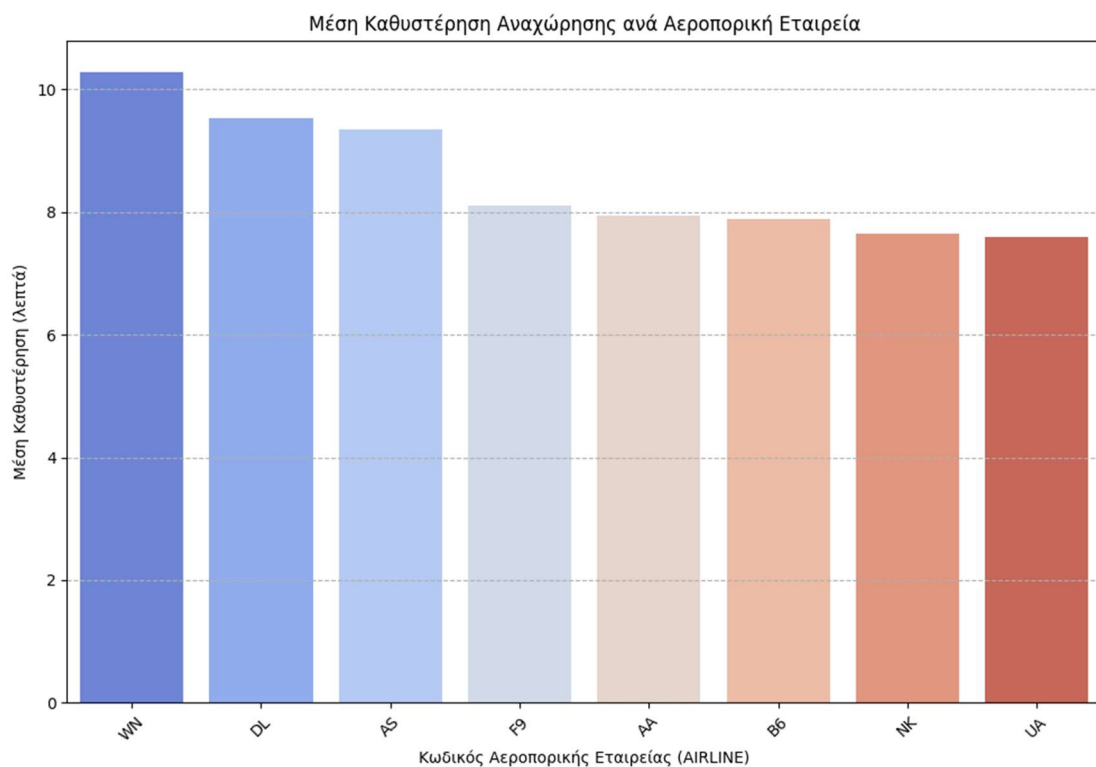
Πίνακας: Μέση Καθυστέρηση ανά Ώρα Αναχώρησης
Πίνακας: Μέση Καθυστέρηση ανά Ώρα Αναχώρησης

Ωρα	Μέση Καθυστερήση (λεπτά)
05:00	9.90
06:00	9.79
07:00	7.35
08:00	9.31
09:00	9.23
10:00	8.21
11:00	7.23
12:00	7.18
13:00	8.96
14:00	7.67
15:00	7.94
16:00	9.03
17:00	8.01
18:00	9.95
19:00	6.39
20:00	8.54
21:00	12.73
22:00	5.99

3)Ανάλυση καθυστέρησης ανά αεροπορική εταιρεία

Ο σκοπός του γραφήματος είναι να συγκριθεί η απόδοση των αεροπορικών εταιρειών ως προς την τήρηση των ωραρίων αναχώρησης.

Δημιουργήσαμε ένα ραβδόγραμμα (seaborn.barplot) που ταξινομεί τις αεροπορικές εταιρείες με βάση την φθίνουσα μέση καθυστέρηση, υπολογίζοντας το πλήθος πτήσεων (FlightCount) και την μέση καθυστέρηση (AvgDelay) ανά AIRLINE.

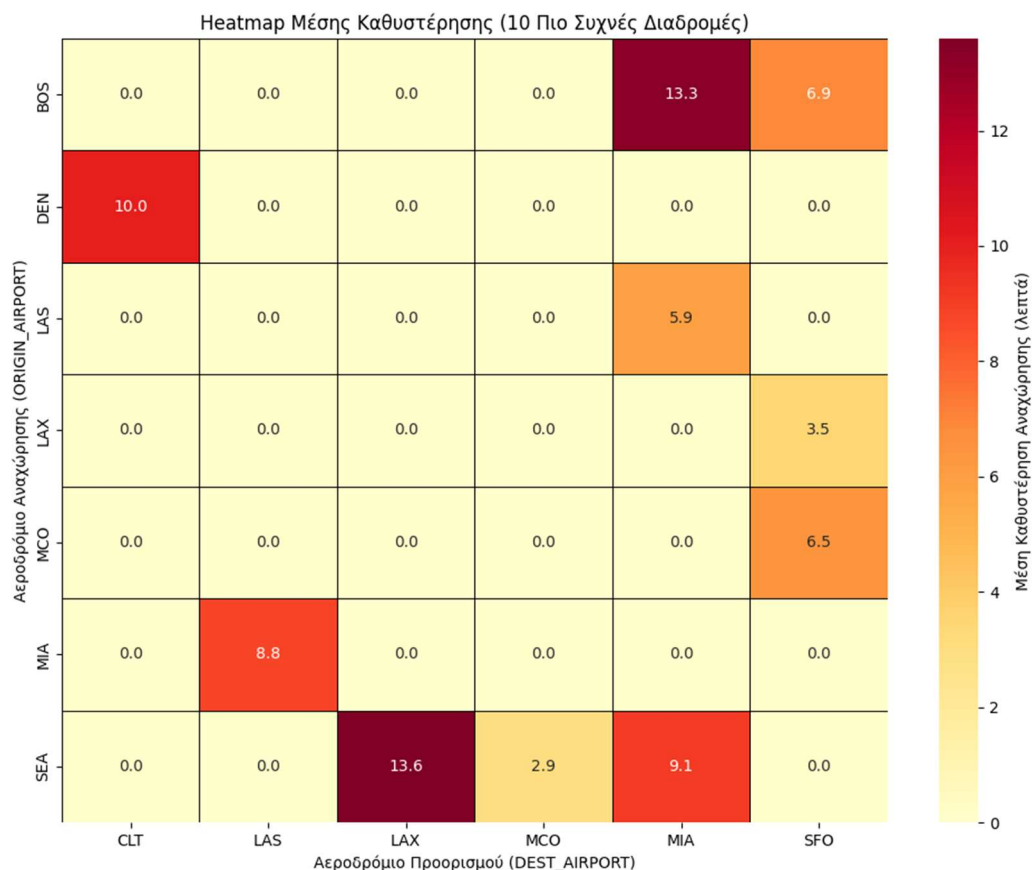


Πίνακας: Μέση Καθυστέρηση και Πλήθος Πτήσεων ανά Αεροπορική Εταιρεία		
AIRLINE	Πλήθος Πτήσεων	Μέση Καθυστέρηση (λεπτά)
WN	236	10.28
DL	265	9.53
AS	235	9.36
F9	237	8.11
AA	229	7.93
B6	246	7.89
NK	258	7.66
UA	263	7.60

4) Heatmap Μέσης Καθυστέρησης

Ο σκοπός του γραφήματος είναι να οπτικοποιηθεί η σχέση μεταξύ των αεροδρομίων αναχώρησης και προορισμού ως προς τη μέση καθυστέρηση εστιάζοντας στις 10 πιο συχνές διαδρομές.

Δημιουργήσαμε ένα Heatmap (seaborn.heatmap) όπου η ένταση του κάθε χρώματος (YlOrRd χρωματική κλίμακα) αντιστοιχεί στην τιμή της μέσης καθυστέρησης. Αυτό επιτρέπει την άμεση αναγνώριση των ζευγών αεροδρομίων με τις υψηλότερες καθυστερήσεις. Υπολογίζετε ο πίνακας pivot με (pandas_df.pivot_table) με ORIGIN_AIRPORT στους δείκτες (index), DEST_AIRPORT στις στήλες (columns) και την AvgDelay ως τιμές.



ΘΕΜΑ 3

Σύγκριση του Spark RDD API με το Spark Dataframe API

Η σύγκριση του Spark RDD API με το Spark Dataframe API δείχνουν σημαντικές διαφορές ως προς την ευκολία ανάπτυξης, τον χρόνο εκτέλεσης, την εκφραστικότητα του κώδικα και τις δυνατότητες βελτιστοποίησης.

Χρόνος Εκτέλεσης και Δυνατότητες Βελτιστοποίησης

Όπως φαίνεται από τους πίνακες που βρίσκονται στην επόμενη σελίδα, το Spark DataFrame API είναι σαφώς ταχύτερο από το Spark RDD API αφού ο μέσος συνολικός χρόνος να είναι **σημαντικά μικρότερος** 0.1625 sec έναντι 1.3687 sec.

Όπως αναφέρεται και στις διαφάνειες της διάλεξης 4.1, η διαφορά αυτή οφείλεται και στην ύπαρξη του Catalyst Optimizer στο DataFrame:

Τα RDDs ήταν ισχυρές αλλά "τυφλές": δεν γνώριζαν το σχήμα. Τα DataFrames εισάγουν semantic awareness - το Spark καταλαβαίνει το "τι" κάνουμε, όχι μόνο το "πώς". Έτσι, η εκτέλεση μπορεί να γίνει ταχύτερη και πιο αποδοτική.

RDD API: Ο προγραμματιστής είναι υπεύθυνος για τη βελτιστοποίηση, καθώς το RDD λειτουργεί σε χαμηλότερο επίπεδο και δεν γνωρίζει το σχήμα των δεδομένων (schema-less).

DataFrame API: Γνωρίζει το σχήμα των δεδομένων (schema-aware) και είναι ικανό να εφαρμόσει διάφορες βελτιστοποιήσεις όπως αυτές που αναφέρονται στην διάλεξη 4.1 pushdown, projection pruning, SQL queries, lazy evaluation και Tungsten Engine του Catalyst Optimizer. Αυτό έχει ως αποτέλεσμα την εκτέλεση λιγότερων και πιο αποδοτικών βημάτων, μειώνοντας αρκετά τον χρόνο εκτέλεσης.

Εκφραστικότητα Κώδικα και Ευκολία Υλοποίησης

RDD API (Χαμηλό Επίπεδο): Κατά την υλοποίηση του κώδικα απαιτείται η συγγραφή custom συναρτήσεων όπως η parse_line και η χρήση πιο δύσκολων μετασχηματισμών όπως το combineByKey. Έτσι αυξάνεται η πολυπλοκότητα του κώδικα και ο χρόνος ανάπτυξης.

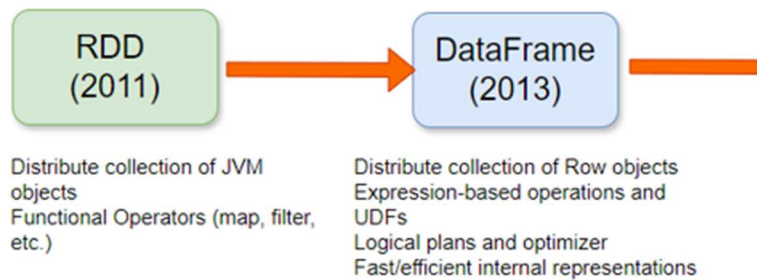
DataFrame API (Υψηλό Επίπεδο): Ο κώδικας είναι πολύ πιο συνοπτικός και ευανάγνωστος. Η χρήση συναρτήσεων όπως groupBy() και orderBy() επιτρέπει στον προγραμματιστή να εστιάσει στο "τι" θέλει να υπολογίσει, αφήνοντας το "πώς" στον Spark (Διάλεξη 4.1 σελίδα 2). Αυτό κάνει την ανάπτυξη ευκολότερη και εκφραστικότερη αποφεύγοντας τα λάθη.

Καταλληλότερη για Μεγάλα Σύνολα Δεδομένων

Από τα αποτελέσματα προκύπτει ότι το Spark Dataframe API είναι καταλληλότερο για μεγάλα σύνολα δεδομένων και αυτό γιατί:

Ανώτερη Απόδοση: Η καλύτερη απόδοση οφείλεται στον Catalyst Optimizer και στο Tungsten Engine, τα οποία διασφαλίζουν ότι οι λειτουργίες εκτελούνται με τον βέλτιστο δυνατό τρόπο κάτι που είναι σημαντικό όταν το μέγεθος των δεδομένων αυξάνεται.

Μειωμένη Ανάπτυξη/Συντήρηση: Ο απλός και εκφραστικός κώδικας οδηγεί σε λιγότερο χρόνο ανάπτυξης και ευκολότερη συντήρηση γεγονός που σημαίνει χαμηλότερο κόστος λειτουργίας.



.(Διάλεξη 3.2)

RDD (2011): Προγραμματιστικά operators (map/filter), πλήρης έλεγχος, αλλά χειροκίνητο optimization .(Διάλεξη 3.2)

DataFrame (2013): Πίνακας με schema – ο Catalyst Optimizer αναλαμβάνει optimization & query planning. (Διάλεξη 3.2)

ΧΡΟΝΟΙ ΕΚΤΕΛΕΣΗΣ ΓΙΑ RDD API

Πίνακας Χρόνων Εκτέλεσης

Εκτέλεση	Συνολικός Χρόνος (sec)	Χρόνος Πρώτου Action (sec)
1	1.5394	0.4083
2	1.4089	0.3306
3	1.3853	0.2727
4	1.3119	0.2909
5	1.3065	0.3128

--- Μέσοι Χρόνοι ---

Μέσος Συνολικός Χρόνος	**1.3687 sec**
Μέσος Χρόνος Πρώτου Action	**0.3114 sec**

ΧΡΟΝΟΙ ΕΚΤΕΛΕΣΗΣ ΓΙΑ Dataframe API

Πίνακας Χρόνων Εκτέλεσης (DataFrame API)

Εκτέλεση	Συνολικός Χρόνος (sec)	Χρόνος Πρώτου Action (sec)
1	1.2212	1.0115
2	0.2209	0.2891
3	0.1213	0.2303
4	0.0955	0.2240
5	0.1454	0.1686

--- Μέσοι Χρόνοι (3 ενδιάμεσες εκτελέσεις) ---

Μέσος Συνολικός Χρόνος	**0.1625 sec**
Μέσος Χρόνος Πρώτου Action	**0.2478 sec**

Περιβάλλον Ανάπτυξης και Εξωτερικές Πηγές

Περιβάλλον Ανάπτυξης

Η ανάπτυξη και εκτέλεση του κώδικα Spark RDD API και Spark DataFrame API πραγματοποιήθηκε στο Colab.

Χρήση Τεχνητής Νοημοσύνης (Gemini)

Στην εργασία χρησιμοποίησα το Gemini του Colab ως βοηθητικό εργαλείο στα ακόλουθα σημεία:

Μορφοποίηση αποτελεσμάτων: Το χρησιμοποίησα στις εκτυπώσεις για τη μετατροπή των τελικών αποτελεσμάτων (πινάκων, χρόνων εκτέλεσης) σε ευανάγνωστη και δομημένη μορφή προκειμένου να βελτιώσω την παρουσίαση.

Συμβουλευτικά κατά την υλοποίηση: : Το χρησιμοποίησα για καθοδήγηση για την δημιουργία του heatmap (Θέμα 5, Ερώτημα c) καθώς η γνώση μου για τη χρήση της λειτουργίας `pivot_table` και της βιβλιοθήκης `seaborn` δεν ήταν πλήρης

Διόρθωση και βελτιστοποίηση κώδικα: Το χρησιμοποίησα κατά τη εκτέλεση του κώδικα για την αναγνώριση και τη διόρθωση συντακτικών ή λογικών σφαλμάτων που προέκυπταν.